

TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG (DAU)

Chương 1: Ôn nền tảng và OOP trong Java

Bài giảng · Chương 1

Học thật · làm thật · code thật

TS. Đỗ Phúc Hảo – Giảng viên *Lecturer*

Khoa Công nghệ Thông tin, Trường Đại học Kiến trúc Đà Nẵng

haodp.ai@gmail.com · 0365 674 592 · Đà Nẵng, 2026

Nội dung buổi học *Lecture outline*

1. Ôn nhanh nền tảng *recap*
- Gói, kiểu dữ liệu, điều khiển

2. Lớp và đối tượng *class*
- Trường, hàm dựng, `static`, `final`

3. Giao tiếp *interface*
- `default`/`static`, so lớp trừu tượng

4. Lớp ẩn danh *anonymous*
- Tiền đề của Lambda

5. Mảng *array*
- Một chiều, đa chiều, `Arrays`

Chương này không dạy lại Java từ đầu: điểm nhanh nền tảng rồi củng cố tư duy hướng đối tượng, làm móng cho các chương sau.



1

Ôn nhanh nền tảng Java

package, types, control flow

Cấu trúc một chương trình Java *Program structure*

DAU

```java
package vn.dau.cntt; // declared first

public class ChaoMung { // name matches file
   public static void main(String[] args) {
      System.out.println("Xin chao Java");
   }
}
```

`.java` → `javac` → mã byte *bytecode* `.class` → máy ảo *JVM* chạy.

GHI NHỚ · KEY POINT

Tên lớp công khai phải trùng tên tệp. Mọi mã đều nằm trong lớp.

Gói và hệ kiểu dữ liệu *Package and types*

Gói *package*

- Không gian tên, gom lớp liên quan
- Tên thường, tên miền đảo: `vn.dau.cntt`
- `import` để gọi bằng tên ngắn

Hai nhóm kiểu

- Nguyên thủy *primitive*: 8 kiểu, giữ giá trị
- Tham chiếu *reference*: lớp, mảng, giao tiếp; giữ địa chỉ
- Lớp bao *wrapper*: `Integer`, `Double`

LỖI HAY GẶP · PITFALL

So sánh đối tượng bằng `==` là so địa chỉ, không so nội dung. Dùng `equals`, nhất là với `String`.

Cấu trúc điều khiển *Control flow*

Giữ nguyên như C

- Rẽ nhánh: `if/else`, `switch`
- Lặp: `for`, `while`, `do-while`
- Điều phối: `break`, `continue`

Java hiện đại cho viết `switch` dạng biểu thức trả về giá trị. Chi tiết để dành chương 6.

GHI NHỚ · KEY POINT

Phần điều khiển ít thay đổi nhất so với cái bạn đã quen, nên ta đi nhanh để dành sức cho phần hướng đối tượng.

2

Lớp và đối tượng

fields, constructors, static, final

Lớp và đối tượng *Class and object*

ĐỊNH NGHĨA · DEFINITION

Lớp *class* là khuôn mẫu mô tả trạng thái và hành vi. **Đối tượng** *object* là một thể hiện cụ thể, sinh ra bằng new, có vùng nhớ riêng.

Trường *field*: biến mô tả trạng thái.
Phương thức *method*: hàm mô tả hành vi.

Hàm dựng *constructor*: trùng tên lớp, không kiểu trả về, chạy một lần khi đối tượng ra đời.

Trường, hàm dựng và this *Fields and this*

DAU

```java
public class SinhVien {
    private String hoTen; // instance field
    private double diem;

    public SinhVien(String hoTen, double diem) {
        this.hoTen = hoTen; // this vs parameter
        this.diem = diem;
    }
    public SinhVien(String hoTen) { // overload
        this(hoTen, 0.0); // delegates
    }
}
```

GHI NHỚ · KEY POINT

`this` trỏ tới đối tượng hiện tại, phân biệt trường với tham số trùng tên.

Nạp chồng *overload* nhiều hàm dựng, gọi chéo bằng `this(...)`.

Bốn mức truy cập *Access levels*

Mức	Cùng lớp	Cùng gói	Lớp con	Mọi nơi
`private`	có	không	không	không
mặc định *package-private*	có	có	không	không
`protected`	có	có	có	không
`public`	có	có	có	có

GHI NHỚ · KEY POINT

Tinh thần che giấu *encapsulation*: để trường ở mức `private`, mở hành vi qua phương thức công khai.

static và final *static and final*

DAU

```java
public class TaiKhoan {
   public static final double LAI_SUAT = 0.05;
   private static int soLuong = 0; // counter
   private final String maSo; // set once

   public TaiKhoan(String maSo) {
      this.maSo = maSo;
      soLuong++; // count every new object
   }
}
```

`static`: thuộc về lớp, mọi thể hiện dùng chung một bản.

`final`: gán một lần; phương thức không cho ghi đè; lớp không cho kế thừa.



3

Giao tiếp *interface*

contract, default & static, vs abstract

Giao tiếp là gì *What is an interface*

ĐỊNH NGHĨA · DEFINITION

Giao tiếp *interface* là hợp đồng mô tả tập hành vi mà lớp cài đặt phải cung cấp, nhưng không quy định cách cài. Định ra cái *gì* cần làm, lớp quyết định làm *thế nào*.

- Khai báo bằng `interface`, liệt kê phương thức trừu tượng
- Lớp dùng `implements` và phải hiện thực mọi phương thức
- Mã gọi chỉ phụ thuộc hợp đồng, nền tảng của đa hình *polymorphism*

Khai báo và cài đặt *Declare and implement*

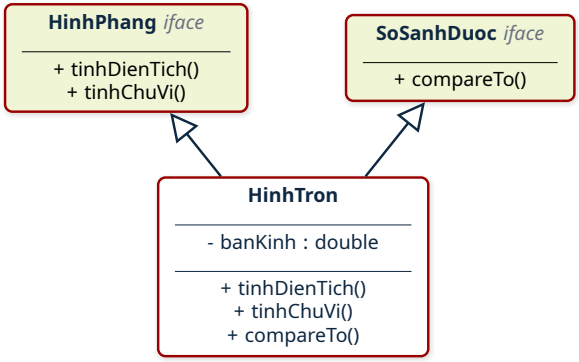
DAU

```java
public interface HinhPhang { // a contract
   double tinhDienTich(); // abstract, no body
   double tinhChuVi();
}

public class HinhTron implements HinhPhang {
   private final double banKinh;
   public HinhTron(double r) { this.banKinh = r; }
   @Override public double tinhDienTich() { return Math.PI * banKinh * banKinh; }
   @Override public double tinhChuVi() { return 2 * Math.PI * banKinh; }
}
```

Đa giao tiếp *Multiple interfaces*

- Lớp cài được nhiều giao tiếp, cách nhau bởi dấu phẩy sau `implements`
- Giải bài toán đa kế thừa *hành vi*, tránh rối của đa kế thừa *trạng thái*
- Mũi tên tam giác rỗng: quan hệ cài đặt



Phương thức default và static *default and static methods*

DAU

```java
public interface HinhPhang {
   double tinhDienTich();
   double tinhChuVi();

   default String moTa() { // body in iface
      return "Dien tich = " + tinhDienTich();
   }
   static HinhPhang rong() { // utility
      return new HinhTron(0);
   }
}
```

`default`: cài sẵn, lớp dùng ngay; thêm hành vi mới mà không phá mã cũ.

`static`: hàm tiện ích gắn giao tiếp, gọi qua tên giao tiếp.

Giao tiếp và lớp trừu tượng *Interface vs abstract class*

DAU

Tiêu chí	Giao tiếp *interface*	Lớp trừu tượng *abstract class*
Từ khoá	`implements`	`extends`
Số lượng	cài nhiều giao tiếp	chỉ kế thừa một lớp
Trường thể hiện	không có trạng thái	có trường đầy đủ
Hằng số	ngầm `public static final`	mọi loại trường
Phương thức có thân	chỉ `default`, `static`	tự do, kèm trừu tượng
Hàm dựng	không có	có
Ý nghĩa	một khả năng, vai trò	một loại thực thể có cốt lõi chung

GHI NHỚ · KEY POINT

Chọn giao tiếp cho một *khả năng* nhiều loại có thể có. Chọn lớp trừu tượng khi nhiều lớp con chia sẻ cài đặt và trạng thái chung.



4

Lớp ẩn danh

anonymous class, gateway to Lambda

Lớp ấn danh là gì *Anonymous class*

DAU

ĐỊNH NGHĨA · DEFINITION

Lớp ấn danh *anonymous class* là lớp không tên, khai báo và tạo thể hiện ngay tại chỗ. Dùng khi chỉ cần đúng một thể hiện duy nhất.

- Viết hẳn một lớp riêng chỉ để dùng một lần là thừa
- Cú pháp: `new` kèm tên giao tiếp/lớp, mở thân ngay sau ngoặc
- Vừa khai báo vừa khởi tạo trong một biểu thức

Cài đặt giao tiếp bằng lớp ẩn danh *Anonymous example*

DAU

```java
public interface XuLy {
   void chay(); // single abstract method
}

XuLy viec = new XuLy() { // anonymous class
   @Override public void chay() {
      System.out.println("Dang chay tac vu");
   }
};
viec.chay();
```

GHI NHỚ · KEY POINT

`XuLy` chỉ có một phương thức trừu tượng: giao tiếp hàm *functional interface*. Khối này về sau viết gọn thành Lambda ở chương 2.

LỖI HAY GẶP · PITFALL

Lớp ẩn danh chỉ truy cập biến cục bộ `final` hoặc thực chất không đổi *effectively final*.



5

Mảng *array*

one-dim, multi-dim, Arrays utility

Mảng là gì *What is an array*

ĐỊNH NGHĨA · DEFINITION

Mảng *array* là đối tượng chứa dãy phần tử cùng kiểu, truy cập qua chỉ số nguyên từ 0. Kích thước cố định khi tạo; trường `length` cho số phần tử.

Mỗi biến mảng là một tham chiếu trỏ tới khối phần tử liên tiếp trên vùng nhớ động *heap*.

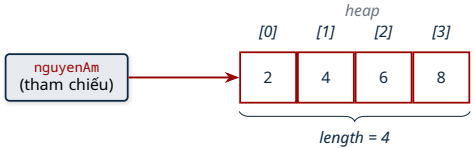
Phần tử chưa gán nhận giá trị mặc định: `0` cho số, `null` cho kiểu tham chiếu.

Mảng một chiều và for-each *One-dimensional array*

DAU

```java
int[] diem = new int[5]; // five ints, default 0
diem[0] = 8; diem[1] = 6; // assign by index
int[] nguyenAm = {2, 4, 6, 8}; // array literal

int tong = 0;
for (int x : nguyenAm) { // for-each: read each
   tong += x;
}
```



Mảng đa chiều *Multidimensional array*

DAU

```java
int[][] bang = new int[3][4]; // 3 rows, 4 cols
for (int i = 0; i < bang.length; i++) {
   for (int j = 0; j < bang[i].length; j++) {
      bang[i][j] = i * 4 + j;
   }
}
// jagged: rows differ in length
int[][] rangCua = { {1}, {2, 3}, {4, 5, 6} };
```

Java cài mảng đa chiều bằng *mảng của mảng*.

Mỗi tham chiếu trỏ một mảng con, nên các hàng có thể khác độ dài: mảng răng cưa *jagged array*.

Lớp tiện ích *Arrays utility*

DAU

```java
import java.util.Arrays;

int[] a = {5, 2, 9, 1};
Arrays.sort(a); // in place: {1,2,5,9}
int vt = Arrays.binarySearch(a, 5); // on sorted
int[] b = Arrays.copyOf(a, 6); // grow, pad 0
System.out.println(Arrays.toString(a)); // [1,2,5,9]
```

LỖI HAY GẶP · PITFALL

`binarySearch` chỉ đúng khi mảng đã sắp xếp. Trên mảng chưa sắp, trả giá trị vô nghĩa mà không báo lỗi.

Mảng có kích thước cố định. Cần tập co giãn theo thời gian chạy, hãy nghĩ tới khung tập hợp *Collections* ở chương 3, ví dụ `ArrayList`.

6

Tóm tắt và bài tập

summary and homework

Tóm tắt chương *Chapter summary*

DAU

Nền tảng
Gói, hai nhóm kiểu, điều khiển như C. So nội dung bằng `equals`, không `==`.

Giao tiếp
Hợp đồng, đa giao tiếp, `default`/`static`; ưu tiên giao tiếp cho một khả năng.

Lớp & đối tượng
Trường, hàm dựng, `this`; `static` dùng chung, `final` đóng băng; che giấu bằng `private`.

Lớp ẩn danh & mảng
Một thể hiện tại chỗ, tiền đề Lambda; mảng cố định, dùng `Arrays`.

Bài tập về nhà *Homework*

DAU

1. Phân biệt biến nguyên thủy và biến tham chiếu. Vì sao so chuỗi bằng `==` có thể sai dù nội dung giống nhau?
2. Viết lớp `SinhVien`: trường `private`, hàm dựng đầy đủ và nạp chồng, phương thức đọc trường, bộ đếm `static` tổng số sinh viên.
3. Khai báo `HinhPhang` (diện tích, chu vi, `default moTa`); cài `HinhChuNhat`, `HinhTron`; `main` tạo mảng và in mô tả bằng `for-each`.
4. Dùng lớp ấn danh tạo thể hiện `XuLy` in thông báo, rồi gọi. Giải thích vì sao viết gọn được thành Lambda.
5. Tạo mảng `int[3][4]` điền `i*4+j`; dùng `Arrays` in từng hàng, sắp xếp một hàng và tìm kiếm nhị phân.

GHI NHỚ · KEY POINT

Khi nào dùng giao tiếp, khi nào lớp trừu tượng? Nêu một tình huống thiết kế cụ thể cho mỗi lựa chọn.

CẢM ƠN

Hỏi đáp · Thảo luận

TS. Đỗ Phúc Hảo

haodp.ai@gmail.com · 0365 674 592 · Khoa CNTT, ĐH Kiến trúc Đà Nẵng